Exploiting Cross-Site Scripting to Capture Passwords

1. Overview

Field	Details
Vulnerability	Cross-Site Scripting (XSS)
Category	Client-Side Attack
Attack Objective	Password Exposure
Attack Technique	Malicious JavaScript Injection
Severity	High to Critical, depending on exposure
Lab Platform	PortSwigger Web Security Academy

2. Description

Cross-Site Scripting can be exploited to compromise sensitive user information when malicious JavaScript executes within a trusted application's origin.

If an application contains an XSS vulnerability, an attacker may manipulate page elements, monitor user interactions, or interfere with authentication forms.

Password capture attacks exploit this behavior by targeting credentials entered into vulnerable webpages.

This can expose user credentials if an attacker-controlled script is able to access password input fields or intercept relevant form interactions.

3. Root Cause

The primary weakness is an exploitable XSS vulnerability that allows untrusted JavaScript to execute within the application's origin.

Potential contributing factors include:

- Unsafe handling of user-controlled input.
- Missing context-aware output encoding.
- Insufficient HTML sanitization.
- Lack of effective client-side script restrictions.
- Inadequate protection of authentication-related interfaces.

4. Proof of Concept (PoC)

Testing environment: Authorized PortSwigger Web Security Academy lab.

1. Open the assigned lab.
2. Identify the XSS injection point.
3. Determine whether JavaScript execution is possible.
4. Inspect the application's authentication interface.
5. Examine whether password fields and form interactions are exposed to scripts executing in the same origin.
6. Use a dedicated test account and dummy credentials in the authorized lab.
7. Verify whether the injected script can interact with the test authentication interface.
8. Record the result without collecting or transmitting real user credentials.

Example password field for security analysis:

```
None
<input
    type="password"
    name="password"
    autocomplete="current-password"
>
```

This illustrates an authentication input field and is not an exploitation payload.

5. Security Impact

Successful exploitation may result in:

- Exposure of user-entered passwords.
- Unauthorized access to user accounts.
- Credential reuse attacks against other services.
- Account compromise.
- Exposure of sensitive personal information.
- Potential compromise of privileged accounts.

The actual impact depends on the application's authentication design and the permissions of affected users.

6. Recommended Remediation

- Eliminate XSS vulnerabilities through contextual output encoding.
- Avoid unsafe HTML and JavaScript injection.
- Use secure templating frameworks.
- Apply strict input handling and HTML sanitization where required.
- Implement a restrictive Content Security Policy.
- Use secure authentication form handling.
- Protect credentials in transit using HTTPS.
- Implement multi-factor authentication.
- Monitor suspicious authentication activity.
- Never expose credentials in URLs, logs, or unnecessary client-side storage.

7. Verification

1. Retest the original XSS injection point.
2. Confirm that injected JavaScript cannot execute.
3. Verify that authentication forms operate securely.
4. Test with dummy credentials in an authorized environment.
5. Confirm that sensitive form data is not unnecessarily exposed.
6. Review browser security policies and authentication behavior.

8. References

- PortSwigger: Exploiting XSS
- PortSwigger: Cross-Site Scripting
- OWASP: Authentication Cheat Sheet
- OWASP: XSS Prevention Cheat Sheet

9. Disclaimer

This report is intended for educational purposes and authorized security assessments. Password testing must use dedicated test accounts and dummy credentials. Never capture or transmit real users' passwords without explicit authorization.